

CabinGuard V2

语音优先的多轮主动式智能座舱 Agent

一、项目定位

CabinGuard V2 是一个基于 **DeepSeek V4**、**LangChain**、**MCP** 和 **Web 交互界面**构建的主动式智能座舱 Agent。

系统通过语音与用户多轮交流，融合车辆状态、驾驶员状态、天气、导航目的地和座舱状态，主动提供天气播报、路线规划、舒适性调节和安全提醒，并通过独立安全门控限制危险操作。

一句话介绍：

CabinGuard V2 不仅能够听懂驾驶员说什么，还能结合目的地、天气、驾驶状态和历史偏好，主动完成导航与座舱服务，同时避免执行不安全的请求。

二、V2 核心闭环



除了用户主动说话，系统还可以由事件自动触发：

- 车辆启动
- 目的地确定
- 天气变化
- 疲劳程度升高
- 驾驶时间过长

座舱温度异常
↓
主动触发 Agent
↓
生成服务方案
↓
安全检查
↓
语音播报并执行

三、功能优先级

P0：必须完成

1. 按键式语音输入。
2. STT 语音转文字。
3. IndexTTS2 语音播报。
4. DeepSeek V4 多轮对话。
5. 天气 MCP 查询与自动播报。
6. 地图展示、地点搜索和路线规划。
7. 语音设置目的地。
8. 六个高层工具调用。
9. Safety Gate。
10. Web 交互页面。
11. 三个稳定的视频演示场景。

P1：尽量完成

1. 疲劳、注意力、压力和心情模拟。
2. 座椅加热、通风、按摩。
3. LangSmith 调用链追踪。
4. 目的地天气与导航联动。
5. Agent 主动触发。
6. 天气和路线结果缓存。

P2：可选增强

1. EverOS 长期记忆。
2. 用户偏好学习。
3. 情绪化 TTS。
4. 唤醒词和连续监听。
5. 实时逐路口语音导航。

四、语音交互方案

4.1 STT: faster-whisper

推荐使用：

```
麦克风录音
↓
VAD 去除静音
↓
faster-whisper
↓
用户文字
```

`faster-whisper` 是基于 CTranslate2 的 Whisper 实现，支持 CPU/GPU 量化和内置 Silero VAD，比较适合单机 Demo。官方项目也给出了 CPU INT8、GPU FP16 和 VAD 的直接用法。

两天版本不要做持续监听，使用更稳定的：

```
按住说话 / 点击开始录音
点击结束
识别
Agent 响应
TTS 播放
```

采用**半双工交互**：

```
用户说话 → 系统识别 → Agent 思考 → 系统播报
```

暂不实现一边播报、一边识别用户打断，避免回声消除、抢话和音频线程问题影响视频稳定性。

4.2 TTS: IndexTTS2

IndexTTS2 支持零样本音色、情绪控制和中文语音生成，可以让天气、导航和安全提醒使用不同的语气。

建议准备三种播报风格：

场景	TTS 风格
天气和日常导航	自然、友好
疲劳提醒	平静、明确
严重安全风险	严肃、简洁

例如：

天气播报：

“早上好，今天目的地有小雨，气温二十二度，建议携带雨具。”

普通导航：

“路线已经规划完成，预计三十五分钟到达。”

疲劳提醒：

“检测到您的疲劳程度较高，请不要继续长时间驾驶。”

IndexTTS2 官方目前推荐使用 `uv` 管理环境，Windows GPU 加速还涉及对应 CUDA 和可选加速组件。因此应尽早单独验证 TTS，不能等全部功能完成后再接入。

必须保留一个备用 TTS。IndexTTS2 环境或推理速度出现问题时，视频版可以切换到浏览器或系统 TTS，保证主流程不受影响。

五、天气 MCP

5.1 实现方式

不建议随意安装第三方 MCP 市场中的天气服务器。

官方 MCP servers 仓库主要提供少量参考实现，而且明确说明这些服务器主要用于教学和协议示例，不应直接视作生产方案；当前参考列表表中也没有现成天气服务器。

本项目自行编写一个很小的开源天气 MCP：

```
weather-mcp/  
├── server.py  
├── weather_client.py  
├── schemas.py  
└── README.md
```

使用：

```
FastMCP  
  ↓  
Open-Meteo API  
  ↓  
结构化天气结果
```

LangChain 官方提供 `langchain-mcp-adapters`，可把 MCP 工具转换为 LangChain/LangGraph Agent 能直接调用的工具，并支持同时连接多个 MCP Server。

天气 MCP 提供三个内部能力：

```
get_current_weather(location)
get_weather_forecast(location, hours)
get_destination_weather(latitude, longitude)
```

天气结果包含：

```
{
  "location": "上海虹桥站",
  "temperature": 22,
  "apparent_temperature": 23,
  "weather": "light_rain",
  "precipitation_probability": 70,
  "wind_speed": 18,
  "advice": "建议携带雨具"
}
```

Open-Meteo 可以返回当前温度、体感温度、降水、天气代码、风速、风向等字段，适合转成简短的座舱播报。

5.2 自动播报触发

自动播报不由 LLM 随机决定，而由触发引擎控制：

```
车辆启动
→ 播报当前位置天气

目的地确认
→ 播报目的地天气

目的地存在明显降雨或降雪
→ 主动提醒

天气刚刚播报过
→ 30 分钟内不重复播报
```

示例：

“目的地杭州西湖今天下午可能有雨，气温二十五度。已经为您规划路线，预计一小时四十分钟到达。”

六、导航地图与语音导航

6.1 地图技术路线

推荐使用：

地图显示：MapLibre GL JS
地点搜索：Nominatim
路线规划：OSRM
地图数据：OpenStreetMap

MapLibre GL JS 是用于网页和 WebView 的开源地图渲染库；Nominatim 提供基于 OpenStreetMap 的地点搜索；OSRM 提供路线、距离、时长和分步导航信息。

两天版本不自行部署全国地图数据和 OSRM 服务，优先调用可用的开发接口，并把视频中的目的地、坐标、路线和天气结果缓存到本地。

6.2 多轮导航流程

用户：

“带我去虹桥站。”

Agent：

“您是要前往上海虹桥火车站，还是虹桥机场？”

用户：

“火车站。”

Agent：

“已经找到上海虹桥站，预计四十二分钟到达。现在开始导航吗？”

用户：

“开始吧，那里天气怎么样？”

系统执行：

保留上一轮候选目的地
↓
确认上海虹桥火车站
↓
规划路线
↓
查询目的地天气 MCP
↓
地图绘制路线
↓
TTS 合并播报

语音输出：

“导航已开始，全程三十二公里，预计四十二分钟。虹桥站目前二十三度，有小雨，建议下车时携带雨具。”

6.3 两天版导航范围

实现：

- 地点搜索；
- 候选地点确认；
- 起点到终点路线；
- 距离和预计时间；
- 地图绘制；
- 起步导航播报；
- 目的地天气播报。

暂不实现真实 GPS 移动和每个路口的连续导航。可以用一个“模拟车辆前进”按钮，让车辆沿路线移动并播放预设转向提示。

七、六个高层工具

为避免给 Agent 暴露过多相似工具，V2 对模型只暴露六个高层工具。

1. weather_service

```
weather_service(  
    action="current | destination | forecast",  
    location="上海虹桥站"  
)
```

通过 MCP 查询天气。

2. navigation_service

```
navigation_service(  
    action="search | preview | start | cancel",  
    destination="上海虹桥站"  
)
```

内部完成地点搜索、路线规划和地图更新。

3. climate_control

```
climate_control(  
    temperature=23,  
    mode="auto"  
)
```

覆盖原来的 `set_temperature` 。

4. media_control

```
media_control(  
    mode="music | podcast | video | off",  
    volume=25  
)
```

覆盖原来的：

```
set_music_volume  
set_media_mode
```

5. seat_control

```
seat_control(  
    heating=0,  
    ventilation=2,  
    massage=1  
)
```

控制：

- 座椅加热；
- 座椅通风；
- 按摩强度。

6. safety_service

```
safety_service(  
    action="alert | suggest_rest_stop",  
    level="warning",  
    message="检测到疲劳驾驶风险"  
)
```

覆盖原来的：


```
show_safety_alert
suggest_rest_stop
```

原来的 `save_user_preference` 不再直接暴露给 Agent，而放到记忆中间件中。只有用户明确说出“记住这个设置”时，系统才写入偏好。

八、安全门控

六个工具都必须经过同一个 Safety Gate：

```
Agent 生成工具调用
  ↓
Safety Gate
  ↓
ALLOW / MODIFY / CONFIRM / BLOCK
  ↓
工具执行器
```

核心规则：

操作	条件	决策
播放视频	车速大于 0	BLOCK
修改目的地	已经处于导航中	CONFIRM
查询天气	任意状态	ALLOW
规划路线	只读预览	ALLOW
开始新导航	目的地存在歧义	CONFIRM
温度低于 18°C或高于 28°C	任意状态	MODIFY
高速时高强度按摩	车速大于 80	MODIFY
关闭安全提醒	任意状态	BLOCK
保存长期偏好	用户未明确授权	CONFIRM
疲劳程度高	高速驾驶	强制安全提醒

例如：

用户：
“我太困了，放个电影，然后把按摩开到最大。”

Agent：
`media_control(mode="video")`

```
seat_control(message=3)
```

Safety Gate:

视频 → BLOCK

按摩等级 3 → MODIFY 为等级 1

最终播报:

“行驶过程中无法播放视频。我已经开启低强度按摩，但当前疲劳程度较高，建议尽快进入休息区。”

九、心情与驾驶员状态

驾驶员状态统一为结构化数据:

```
{
  "fatigue_level": 0.86,
  "attention_level": 0.43,
  "stress_level": 0.71,
  "mood": "tired",
  "driving_duration_minutes": 135
}
```

两天版本通过 Web 滑块和场景按钮模拟:

正常
开心
疲惫
焦虑
注意力分散
严重疲劳

可以让用户通过语音补充状态:

“我今天心情不太好。”

Agent 将它转换成:

```
{
  "mood": "low",
  "source": "user_statement",
  "confidence": 0.9
}
```

然后提出：

- 播放舒缓音乐；
- 调低音量；
- 开启低强度按摩；
- 使用更平静的 TTS 风格。

LangSmith 的正确定位

LangSmith **不是** 驾驶员情绪或疲劳检测模型。

它用于记录和观察：

- 用户语音识别结果；
- Agent 输入和输出；
- MCP 调用；
- 工具选择；
- Safety Gate 决策；
- 响应耗时；
- 工具错误；
- 多轮上下文是否正确。

LangSmith 官方将其定位为 LLM 应用的 tracing、监控、评估和故障分析平台。

Web 页面可以显示：

```
STT: 1.1 s
Agent: 2.3 s
Weather MCP: 0.5 s
TTS: 2.0 s
Safety Gate: BLOCK
总延迟: 5.9 s
```

这样“驾驶员状态”是 Agent 的输入，而 LangSmith 负责验证 Agent 是否正确使用这些状态。

十、记忆方案

MVP

两天版本首先使用：

```
SQLite + JSON
```

保存少量明确偏好：

```
{
  "preferred_temperature": 23,
  "preferred_music_volume": 25,
  "preferred_seat_massage": 1,
  "home_location": "上海",
  "weather_broadcast_enabled": true
}
```

只记忆稳定、明确且得到用户授权的信息。

EverOS 可选增强

你提到的 EverMind 项目当前对应的开源记忆系统是 **EverOS**。它采用本地优先设计，以 Markdown 作为可读写的记忆源，并使用 SQLite 和 LanceDB 建立索引，适合长期用户画像和跨会话记忆。

但是，其完整服务流程当前还会涉及额外的 LLM、Embedding 和 Rerank 配置。对于只剩两天的 MVP，直接接入可能增加环境和接口风险。

因此推荐：

CabinGuard V2 MVP：
SQLite/JSON

CabinGuard V2.1：
接入 EverOS

视频中可以展示“记忆接口已预留”，但不要让记忆组件拖慢 P0 功能。

十一、Web 交互页面

推荐使用：

后端：FastAPI
Agent：LangChain
前端：Streamlit + 自定义 HTML/JavaScript
地图：MapLibre
通信：HTTP + WebSocket 或轮询

页面采用三栏布局。

左侧：车辆与驾驶员模拟器

当前车速
驾驶时长

疲劳程度
注意力
压力
心情
座舱温度
当前位置

场景按钮：

正常通勤
下雨天出行
语音导航
目的地天气
心情低落
疲劳驾驶
高速危险请求

中间：虚拟座舱与地图

显示：

- 当前路线；
- 起点、终点；
- 剩余距离；
- 预计到达时间；
- 天气卡片；
- 空调温度；
- 媒体状态；
- 座椅状态；
- 安全提醒。

右侧：Agent 面板

显示：

语音识别文字
Agent 回复
多轮对话历史
调用的工具
Safety Gate 结果
MCP 返回结果
LangSmith Trace 状态

底部：语音控制

[开始录音]
[停止录音]
[播放回复]
[中止播报]

状态：

正在录音 / 正在识别 / Agent 思考中 / 正在播报

十二、最终视频演示脚本

建议视频控制在 3~4 分钟。

场景一：车辆启动与主动天气服务

车辆启动，系统读取当前位置和座舱状态。

系统主动播报：

“早上好。当前二十六度，今天有小雨。车内温度偏高，是否为您调到二十三度并开启座椅通风？”

用户：

“可以，通风开到二档。”

系统执行：

```
weather_service  
climate_control  
seat_control
```

场景二：多轮语音导航与目的地天气

用户：

“带我去虹桥站。”

系统：

“您是要去上海虹桥火车站还是虹桥机场？”

用户：

“火车站，顺便看看那里天气。”

系统：

“上海虹桥站目前有小雨，已经规划路线，预计四十二分钟到达。现在开始导航吗？”

用户：

“开始吧。”

地图绘制路线并播放导航语音。

场景三：驾驶员状态与安全拦截

模拟状态：

车速：105 km/h
驾驶时间：145 分钟
疲劳程度：0.90
注意力：0.38

用户：

“我太困了，放个电影，把按摩开到最大。”

Agent 理解用户真实需求是缓解疲劳。

Safety Gate：

播放视频 → BLOCK
最大按摩 → MODIFY
安全提醒 → ALLOW
休息区建议 → ALLOW

系统使用严肃、平静的 TTS：

“行驶过程中不能播放视频。我已经开启低强度按摩，但您目前存在明显疲劳风险。建议前往前方休息区休息。”

这是整个视频最重要的差异化场景。

十三、两天开发安排

第一天上午

- 建立项目目录；
- 连接 DeepSeek V4；
- 实现多轮消息状态；
- 定义六个高层工具；
- 实现 Safety Gate；
- 完成基本文本对话。

第一天下午

- 接入麦克风录音；
- 接入 faster-whisper；
- 接入 IndexTTS2；
- 跑通“语音输入—Agent—语音输出”；
- 准备备用 TTS。

第一天晚上

- 编写天气 MCP；
- 接入天气 API；
- 实现自动天气播报；
- 实现天气结果缓存。

第二天上午

- 接入 MapLibre；
- 实现地点搜索和路线规划；
- 完成语音导航多轮流程；
- 加入目的地天气联动。

第二天下午

- 完善 Web 页面；
- 加入驾驶员状态场景；
- 接入 LangSmith tracing；
- 加入座椅控制；
- 固化三个视频场景；
- 录制备用演示视频。

十四、项目目录

```
cabin-guard-v2/  
├── app.py
```



```
├── backend/
│   ├── api.py
│   └── websocket.py
├── agent/
│   ├── cabin_agent.py
│   ├── prompts.py
│   ├── conversation_state.py
│   └── schemas.py
├── speech/
│   ├── recorder.py
│   ├── stt_service.py
│   ├── tts_service.py
│   └── audio_queue.py
├── mcp_servers/
│   └── weather/
│       ├── server.py
│       └── weather_client.py
├── navigation/
│   ├── geocoder.py
│   ├── route_service.py
│   └── navigation_state.py
├── tools/
│   ├── weather_tool.py
│   ├── navigation_tool.py
│   ├── climate_tool.py
│   ├── media_tool.py
│   ├── seat_tool.py
│   └── safety_tool.py
├── safety/
│   ├── safety_gate.py
│   └── policies.py
├── simulator/
│   ├── vehicle_state.py
│   ├── driver_state.py
│   ├── trigger_engine.py
│   └── scenarios.py
├── memory/
│   ├── memory_store.py
│   └── user_profile.json
├── frontend/
│   ├── dashboard.py
│   └── map_component/
├── logs/
│   └── decision_logger.py
└── tests/
    ├── test_safety.py
    ├── test_weather.py
    └── test_dialogue.py
```

十五、最终验收标准

项目完成时至少应满足：

1. 用户能够通过麦克风与 Agent 交流。
2. Agent 能连续完成三轮以上语音对话。
3. 系统可以主动播报当前位置天气。
4. 可以通过语音设置并确认目的地。
5. 地图能够绘制路线。
6. 能播报目的地天气和预计到达时间。
7. 六个高层工具均能真实触发页面状态变化。
8. 危险工具调用会被 Safety Gate 拦截或修改。
9. 座椅加热、通风、按摩可视化。
10. Web 页面能显示 STT、Agent、MCP、工具和安全日志。
11. 三个核心场景能够稳定重复演示。
12. 即使实时天气或地图接口失败，也能使用缓存数据完成视频。

最终项目表述

CabinGuard V2 是一个语音优先的主动式智能座舱 Agent。它通过 DeepSeek V4 和 LangChain 实现多轮情境理解，通过开源 MCP 获取实时天气，通过地图与导航服务完成目的地规划，并结合驾驶员状态主动调整空调、媒体和座椅。所有工具调用均经过独立安全门控，使座舱既能够主动理解和服务用户，也能够危险请求面前保持克制。